

BuMoChi フルセットアップ手順

BuMoChi

クイックスタート：ワンコマンド・パイプラインランチャー

ランチャーは **BunrakuOSCEncoder** と **BunrakuOSCDecoder** を起動し、監視します。引数なしで実行するとローカル専用パイプラインが起動し、**OscGroupClient** は起動しません。

./PipelineApplications/start_bumochi_pipeline.sh

このコマンドは BuMoChi リポジトリのルートから実行してください。標準のローカルポートが自動的に使用されます。

XR-Animator -> encoder 39537 -> Bmc 57130 -> decoder 39538 -> Godot 39539

ランチャーは XR-Animator、SuperCollider、Godot を起動しません。この3つのアプリケーションは個別に起動してください。SuperCollider のクラスライブラリを再コンパイルして Bmc が **57130** で待ち受けるようにし、XR-Animator の送信先を **39537** に設定し、Godot のアバターレシーバーを **39539** で実行します。

OSCGroups を使用するには、OSC サーバーのアドレスと一意のユーザー名を指定します。

```
./PipelineApplications/start_bumochi_pipeline.sh \  
  --oscsrvr SERVER_ADDRESS \  
  --username PerformerA
```

OSCGroups モードでは、次の追加デフォルト値が使われます。

設定	デフォルト	上書きオプション
グループ名	bumochi	--groupname NAME
ユーザーおよびグループのパスワード	bmc123	--password PASSWORD
サーバーポート	22242	--server-port PORT
ローカルのネットワーク接続用ポート	22243	--local-port PORT

パスワード **bmc123** はリハーサル用の便利なデフォルト値であり、公開ネットワーク向けの安全な認証情報ではありません。外部に公開された OSCGroups サーバーでは別のパスワードを設定してください。

何も起動せず、利用可能なすべてのランチャーオプションを確認するには、次を実行します。

./PipelineApplications/start_bumochi_pipeline.sh --help

ランチャーが起動したすべての補助プロセスを停止するには、そのターミナルで **Control-C** を押します。

ランチャーを **--verbose** 付きで起動した場合、別のターミナルでエンコーダーとデコーダーの出力をリアルタイムに確認できます。

```
tail -f "$TMPDIR/bumochi-pipeline-$USER/encoder.log" \  
        "$TMPDIR/bumochi-pipeline-$USER/decoder.log"
```

ランチャーは Python を非バッファモードで起動するため、新しいログ行は直ちに表示されます。

チートシート：現在の Bmc ポート設定を確認する

VMC モーションキャプチャーアプリケーション（XR-Animator、Waidayo など）と Godot は、パイプラインに必要なポートを使用してアニメーションデータを送受信するように設定する必要があります。ポート番号を確認するには、SuperCollider のクラスライブラリを再コンパイルした後、次を評価してください。

Bmc.help;

デフォルトの概要は次と同等です。

XR-Animator output port: 39537
 Bunraku0SCEncoder output port: 57130
 SuperCollider VMC/OSC input port: 57130
 SuperCollider VMC/OSC output port: 39538
 Bunraku0SCDecoder input port: 39538
 Bunraku0SCDecoder output ports:
 Ishidomaru: 39539

最後の一覧は、Bmc に現在設定されているアバターから生成されます。そのため、アバタールートを追加すると一覧も増えます。ポートを変更した後は、もう一度 **Bmc.help;** を実行してください。

以下のセクションは、パイプラインをカスタマイズする場合、またはエラーを確認する場合にのみ必要です。

詳細

このガイドでは、パイプライン内の各アプリケーションの内部ポート番号を調整したり、問題を診断したりするための情報を提供します。次の下流方向の順序で、6つのアプリケーションを起動して設定する方法を説明します。

No.	アプリケーション	入力／待受ポート	出力／送信先ポート	必要な設定
1	XR-Animator	—	39537	標準 VMC を 127.0.0.1:39537 の Bunraku0SCEncoder に送信します。
2	Bunraku0SCEncoder	39537	57130、必要に応じて 22244	XR-Animator を受信し、ルート情報を持たないフレームを常にローカル Bmc の 57130 に送信します。OSCGroups モードでは、同一のコピーをローカル OscGroupClient の 22244 にも送信します。
3	OscGroupClient	22244	57130	22244 でローカルエンコーダーのフレームを受信し、遠隔共同作業者のフレームを 57130 のローカル Bmc に渡します。また、クライアント固有のローカルポート（例：22243）を経由して OscGroupServer のポート 22242 に接続します。
4	SuperCollider Bmc	/ 57130	39538	ローカルおよび遠隔のルート情報を持たないフレームを受信し、ローカルシーンを合成して、ルート付きフレームを 39538 の Bunraku0SCDecoder に送信します。Ishidomaru のデフォルトのルート付きフレームには、Godot の送信先 39539 が埋め込まれます。
5	Bunraku0SCDecoder	39538	39539	Bmc のルート付きフレームを受信して標準 VMC を再構成し、Ishidomaru のデータを埋め込み先である Godot ポート 39539 に転送します。
6	Godot	39539	—	39539 で待ち受ける Ishidomaru VMC レシーバーを実行し、アニメーションをローカルにレンダリングします。

アプリケーションとポートの流れは次のようになります。

```

1. XR-Animator          --VMC 39537-----> 2. BunrakuOSCEncoder
2. BunrakuOSCEncoder    --frames 57130-----> 4. SuperCollider / Bmc
2. BunrakuOSCEncoder    --frames 22244-----> 3. OscGroupClient
3. OscGroupClient       --remote frames 57130-> 4. SuperCollider / Bmc
4. SuperCollider / Bmc  --routed frames 39538-> 5. BunrakuOSCDecoder
5. BunrakuOSCDecoder    --VMC 39539-----> 6. Godot / Ishidomaru

```

この手順では、デフォルトの単一アバター Ishidomaru 設定を使用します。Godot の Ishidomaru VMC レシーバーは **39539** で待ち受けます。

グローバルにインストールされたランチャーを使用するコマンドを除き、すべてのターミナルコマンドは BuMoChi リポジトリのルートから実行してください。

UDP 待受ポートを所有するのは受信側アプリケーションだけです。Bmc が唯一の受信者であるため、複数のアプリケーションが同じ **57130** の Bmc にパケットを送信しても競合しません。

起動前の準備

1. Python 3 が利用できることを確認します。

`python3 --version`

2. OSCGroups を使って共同作業する場合に限り、セッション主催者から **OscGroupServer** のアドレスと一意のユーザー名を取得します。セッションがデフォルトのグループ **bumochi** とパスワード **bmc123** を使用するのか、別の値を使用するのか確認してください。
3. OSCGroups を使って共同作業する場合に限り、このワークステーションに一意の OSCGroups ユーザー名と、エンコーダーの **--source** 識別子を設定します。上書きしない場合、ランチャーはローカルのネットワーク接続用ポート **22243** を使用します。
4. 以前に起動したエンコーダー、デコーダー、**OscGroupClient** の各プロセスを停止します。必要に応じて、ローカルの待受ポートを確認します。

```

lsof -nP -iUDP:39537
lsof -nP -iUDP:22244
lsof -nP -iUDP:57130
lsof -nP -iUDP:39538
lsof -nP -iUDP:39539

```

1. XR-Animator

1. XR-Animator を起動します。
2. VMC/OSC 出力設定を開きます。
3. VMC の送信先を次のように設定します。

```

Host: 127.0.0.1
Port: 39537

```

4. 全身 VMC 出力を有効にします。

XR-Animator はエンコーダーより先に起動してもかまいません。エンコーダーが待ち受けを開始する前に送信された UDP フレームは破棄されます。

スクリーンショットと詳しい設定手順については、[XR-Animator の VMC 出力ポート](#) および [XR-Animator の設定](#) を参照してください。

2. BunrakuOSCEncoder

BuMoChi リポジトリのルートでターミナルウィンドウを開き、関連するすべてのポートを明示してエンコーダーを起動します。

```

python3 PipelineApplications/BunrakuOSCEncoder.py \
--listen-ip 127.0.0.1 \
--listen-port 39537 \
--bmc-ip 127.0.0.1 \

```

```
--bmc-port 57130 \
--oscgroups-ip 127.0.0.1 \
--oscgroups-port 22244 \
--avatar "Ishidomaru" \
--source "workstation-a-xr-animator" \
--verbose
```

workstation-a-xr-animator を、このワークステーション固有の安定したソース識別子に置き換えてください。このターミナルは開いたままにします。

エンコーダーは、ルート情報を持たない同一のソースフレームを 2 つ生成します。

```
local copy    -> UDP 57130 -> local Bmc
network copy -> UDP 22244 -> local OscGroupClient
```

この起動順序ではエンコーダーが **OscGroupClient** と Bmc より先に起動するため、最初の送信パケットは破棄される場合があります。これらの受信側が起動すると通常の配信が始まります。

グローバルにインストールされたランチャーを利用できる場合、同等のコマンドは次のとおりです。

```
BunrakuOSCEncoder \
--listen-ip 127.0.0.1 \
--listen-port 39537 \
--bmc-ip 127.0.0.1 \
--bmc-port 57130 \
--oscgroups-ip 127.0.0.1 \
--oscgroups-port 22244 \
--avatar "Ishidomaru" \
--source "workstation-a-xr-animator" \
--verbose
```

インストール、オプション、診断手順については、[BunrakuOSCEncoder の起動方法](#)および [BunrakuOSCEncoder のテストと診断](#)を参照してください。

3. OscGroupClient

BuMoChi リポジトリのルートで 2 つ目のターミナルウィンドウを開きます。

OscGroupClient には、次の順序で 9 個の位置引数が必要です。

```
OscGroupClient SERVER_ADDRESS SERVER_PORT LOCAL_TO_REMOTE_PORT INPUT_PORT OUTPUT_PORT
↳ USER_NAME USER_PASSWORD GROUP_NAME GROUP_PASSWORD
```

同梱されている macOS クライアントを次のように起動します。

```
HelperAppsAndExamples/OSCGroups/bin/macos/OscGroupClient \
SERVER_ADDRESS 22242 22243 22244 57130 \
USER_NAME USER_PASSWORD GROUP_NAME GROUP_PASSWORD
```

大文字のプレースホルダーをすべて置き換えてください。例のローカルポート **22243** が使用中の場合、または同じコンピューター上の別のクライアントが使用している場合は、空いているポートに置き換えます。このターミナルは開いたままにします。

アプリケーション側で重要なポートは次のとおりです。

```
OscGroupClient input:  UDP 22244 <- local encoder frames for sharing
OscGroupClient output: UDP 57130 -> remote source frames delivered to local Bmc
```

クライアントの出力先をデコーダーポート **39538** に設定しないでください。遠隔から届いたルート情報を持たないフレームは、ローカルでの選択、合成、録画、生成のため、まず Bmc に入る必要があります。

グローバルにインストールされたクライアントを利用できる場合も、同じ引数を使用します。

```
OscGroupClient \
SERVER_ADDRESS 22242 22243 22244 57130 \
USER_NAME USER_PASSWORD GROUP_NAME GROUP_PASSWORD
```

引数の定義、インストール、確認方法については、[OscGroupClient の起動方法](#)、[OscGroupClient のコマンドライン引数](#)、[OSCGroupClient のポート](#)を参照してください。

4. SuperCollider

1. SuperCollider を起動します。
2. 必要に応じて **Language** → **Recompile Class Library** を選択し、クラスライブラリを再コンパイルします。
3. コンパイル後、Bmc は UDP **57130** で **/bunraku/vmc/frame** を自動的に待ち受けます。
4. 次のブロックを評価して、デフォルトの Ishidomaru ルートを確認し、明示的に復元します。

```
(  
Bmc.avatar(\Ishidomaru).vmcPort_(39539);  
Bmc.decoderPort_(39538);  
Bmc.forwardDecoder_(true);  
Bmc.start(57130);  
Bmc.status;  
)
```

5. ライブ入力モニターを開きます。

```
Bmc.showDispatcherStatus;
```

静的フィールドには次のように表示されます。

```
Listening for '/bunraku/vmc/frame' on port: 57130
```

XR-Animator が送信中であれば、動的な **received** カウントが増加します。この値には、エンコーダーから直接受信したローカルフレームと、**OscGroupClient** を通じて受信した遠隔フレームの両方が含まれる場合があります。

詳細については、[OscGroupClient](#) を待ち受ける SuperCollider のポート、[クラスライブラリの新規コンパイル後における BuMoChi のデフォルト転送パイプライン](#)、[Bmc システム制御](#)を参照してください。

5. BunrakuOSCDecoder

BuMoChi リポジトリのルートで3つ目のターミナルウィンドウを開き、デコーダーを起動します。

```
python3 PipelineApplications/BunrakuOSCDecoder.py \  
--listen-ip 127.0.0.1 \  
--listen-port 39538 \  
--target-ip 127.0.0.1 \  
--allow-target-port 39539 \  
--verbose
```

このターミナルは開いたままにします。デコーダーは Bmc がローカルに合成したルート付きフレームを **39538** で受信し、埋め込まれたアバターの送信先を読み取り、標準 VMC バンドルを再構成して、Ishidomaru のバンドルを Godot の **39539** に送信します。

グローバルにインストールされたランチャーを利用できる場合、同等のコマンドは次のとおりです。

```
BunrakuOSCDecoder \  
--listen-ip 127.0.0.1 \  
--listen-port 39538 \  
--target-ip 127.0.0.1 \  
--allow-target-port 39539 \  
--verbose
```

複数アバターの Godot シーンでは、そのシーンが使用するすべての送信先を許可します。例：

```
python3 PipelineApplications/BunrakuOSCDecoder.py \  
--listen-ip 127.0.0.1 \  
--listen-port 39538 \  
--target-ip 127.0.0.1 \  
--allow-target-port 39539 \  
--allow-target-port 39540 \  
--verbose
```

インストール、オプション、診断については、[BunrakuOSCDecoder の起動方法](#)および [BunrakuOSCDecoder のテストと診断](#)を参照してください。

6. Godot

1. Godot を起動し、使用するアバタープロジェクトを開きます。
2. デフォルトの単一アバターテストでは、**Seed_2_Ishidomaru_C** を開きます。
3. プロジェクトの Ishidomaru VMC レシーバーが UDP **39539** で待ち受けていることを確認します。
4. プロジェクトのシーンを実行します。

Godot は最後に起動してもかまいません。レシーバーが起動する前に送信された VMC バンドルは破棄され、シーンが実行されて待受状態になるとライブアニメーションが始まります。

詳しい確認方法と設定手順については、[Godot のアバター固有待受ポート](#)を参照してください。

2 体のアバターを均一に扱う E プロジェクトでは、2 つの明示的なレシーバーノードを使用します。

アバター	レシーバーノード	UDP ポート	ボディトラッカー名	フェイストラッカー名
Mother Ishidomaru	MotherVMCTracker IshidomaruVMCTracker	39539 39540	/vmc/mother_body_tracker /vmc/ishidomaru_body_tracker	/vmc/mother_face_tracker /vmc/ishidomaru_face_tracker

このシーンを使用する場合は、Bmc のアバタールートも対応するように設定し、両方のポートを許可してデコーダーを起動します。

```
Bmc.avatar(\Ishidomaru).vmcPort_(39540);  
Bmc.addAvatar(\Mother, "Mother").vmcPort_(39539);
```

明示的レシーバー構成については、[Godot における複数アバタープロジェクトのポート番号設定](#)を参照してください。

パイプライン全体を確認する

1. XR-Animator に、VMC 出力が **127.0.0.1:39537** に対して有効であることが表示されます。
2. エンコーダーの **received**、**bmc_sent**、**oscggroups_sent** の各カウンタが増加します。
3. **OscGroupClient** に、サーバーとグループへの登録成功が表示されます。
4. **Bmc.showDispatcherStatus** に **running: true**、ポート **57130**、および増加する **received** カウントが表示されます。
5. デコーダーに、**39538** で受信したフレームと **39539** への VMC 出力が表示されます。
6. Godot 内で Ishidomaru が動きます。

SuperCollider から Godot までの最終経路だけをテストするには、次を評価します。

```
Bmc.sendCalibrationFrame;
```

終了

アプリケーションを逆順に停止します。

1. 実行中の Godot シーンを停止します。
2. **Control-C** で **BunrakuOSCDecoder** を停止します。
3. SuperCollider で **Bmc.stop;** を評価します。
4. **Control-C** で **OscGroupClient** を停止します。
5. **Control-C** で **BunrakuOSCEncoder** を停止します。
6. XR-Animator の VMC 出力を無効にするか、XR-Animator を終了します。